

AI 텍스트 요약 서비스를 위한 AWS EKS · GitOps 인프라 프로젝트

운영에서 구축으로 : IaC, GitOps, 관측성을 한 사이클로 직접 돌려봤습니다

고혁배
(AWS · Terraform · EKS · ArgoCD)

2026.07 ~ 2026.08 (구축)
수명주기 재현 5회 (7~8월)

목차

CONTENTS

01 프로젝트 개요

02 프로젝트 요약 · 성과

03 아키텍처 흐름도 구성

04 기술스택 활용

05 애플리케이션 구성

06 시연 과정 (GitOps · 알람 · HPA)

07 트러블슈팅 · 재점검

08 핵심 설계 의사결정

09 마무리

01. 프로젝트 개요

“관제하고 운영하던 클라우드를, 이번에는 직접 설계하다”

- 통합관제 솔루션 (ZENIUS) 기술지원 : KT Cloud(PPP) 와 온프레미스에 관제 시스템을 구축하고 Linux·PostgreSQL 장애를 분석
- AI 클라우드 운영 (NHN·Naver Cloud): Docker 이미지 표준화로 개발 환경 제공 시간 40% 단축
- 운영하며 느낀 수작업의 한계를 , AWS SAP 를 준비하며 익힌 설계 지식으로 직접 구축하고 검증해 봤습니다

IaC

(Infrastructure as Code)

Terraform 으로 리소스 78 개 코드화
apply 한 번으로 같은 환경 재현

GitOps

(CI/CD)

push 한 번으로 빌드부터 배포까지
ArgoCD 가 drift 감지 · 자동 복구

보안

(Security)

OIDC 키리스 인증 · IRSA 최소 권한
비밀값 Secret 분리 주입

관측 · 확장

(Observability · HPA)

Slack 알람 실제 수신 확인
HPA 2→6→2 증감 확인

02. 프로젝트 요약·성과

목표	AI 텍스트 요약 서비스를 EKS 에 GitOps 로 배포하고 , 관측성과 오토스케일링까지 구축
기간	2026.07 ~ 08 (실작업 약 8 일 , 주말·저녁 활용)
핵심 기술	Terraform · EKS · RDS(PostgreSQL 17) · GitHub Actions(OIDC) · ArgoCD · Kustomize · HPA · Prometheus/Grafana · CloudWatch
규모	Terraform 리소스 78 개 · 트러블슈팅 기록 7 건 · ADR 3 건 · 수명주기 재현 5 회

전 항목 검증 완료

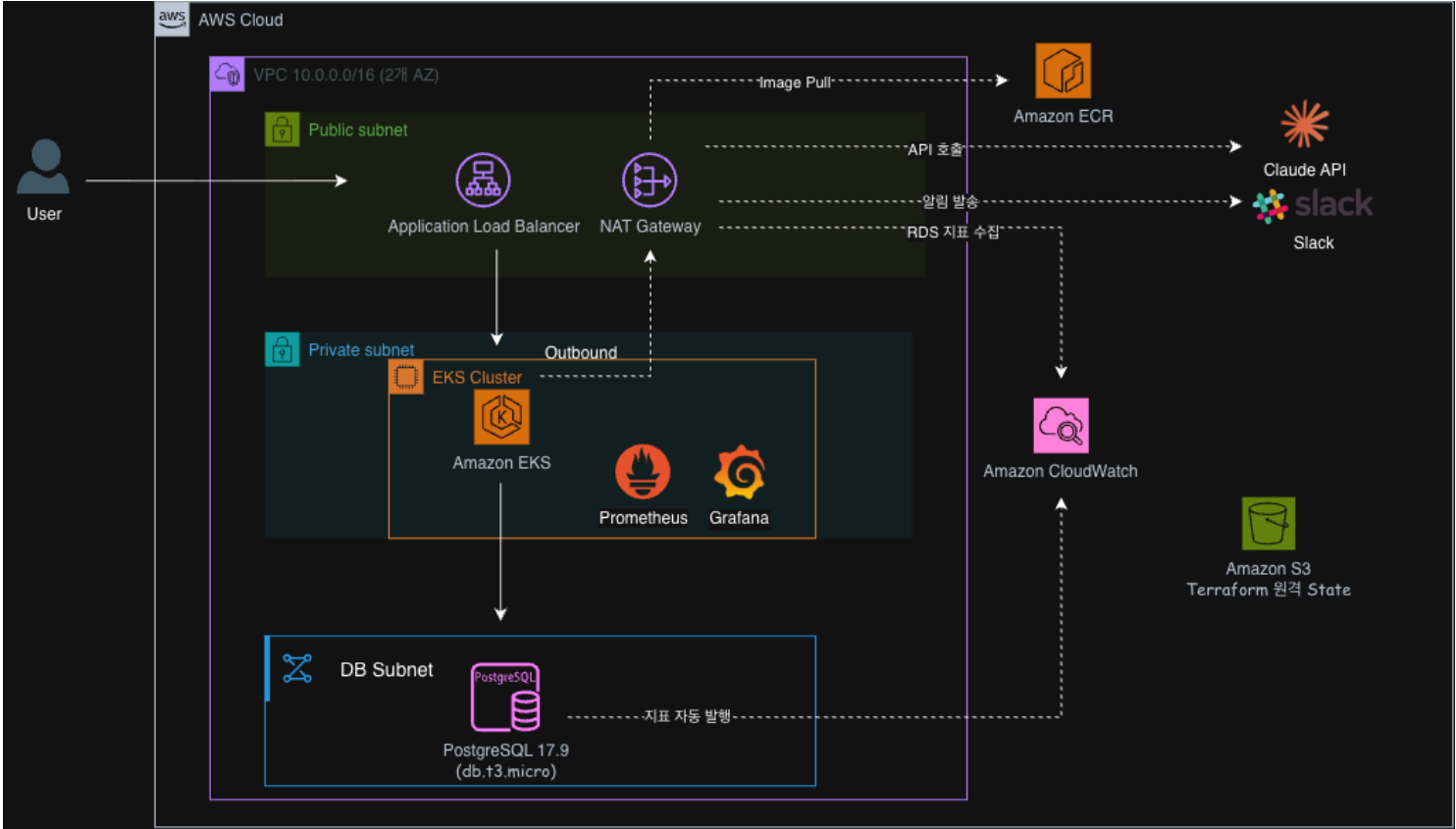
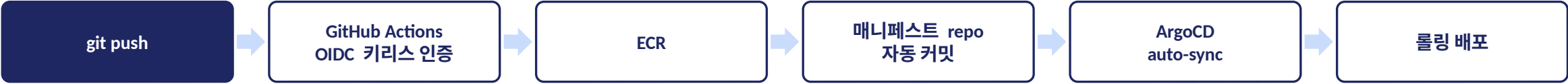
✓ 서비스 E2E

✓ GitOps 자동배포

✓ HPA 2→6→2 실증

✓ Slack 알람 수신

03. 아키텍처 흐름도 구성



설계 포인트

- 1 액세스 키 없는 CI 인증
GitHub OIDC → AWS Role
- 2 파드 단위 최소 권한
IRSA 3 종 : ALB Controller · EBS CSI · CloudWatch exporter
- 3 상태 / 무상태 계층 분리
3 계층 서브넷 설계

런타임 구성도: 3 계층 서브넷 격리, NAT 단일 아웃바운드, RDS 지표는 CloudWatch 를 거쳐 EKS 로

03. 아키텍처 흐름도 구성 - 핵심 설계 의도

< 핵심 설계 의도 >

- 1) 외부 진입점은 ALB 하나로 . target-type: ip 로 파드에 직접 라우팅해 홉을 줄임
- 2) EKS·RDS 는 Private/DB 서브넷에 두고 , RDS 는 노드 SG 를 참조해 5432 만 허용
- 3) 기반 자원은 전부 Terraform 으로 . 서브넷 태그까지 코드에 적고 , state 밖 자원 (ALB·EBS) 은 종료 절차로 회수해 destroy 후 잔여물 13 개 항목을 직접 확인
- 4) 부하는 HPA 가 , 배포는 ArgoCD 가 . replicas 소유권을 나눠 두 도구가 충돌하지 않게 함

04. 기술스택 활용

클라우드

AWS (ap-northeast-2)

클라우드 인프라 및
서비스 운영 기반

컨테이너·오케스트레이션

Docker · ECR · EKS · HPA

컨테이너 빌드부터 배포,
자가치유 · 자동 확장까지

인프라 코드화 (IaC)

Terraform · S3 (원격 state)

리소스 78 개 코드 정의
state 는 S3 에 두고 destroy·apply 반복

CI/CD (GitOps)

GitHub Actions · ArgoCD · Kustomize · Helm

push 기반 자동 배포와
drift 자동 복구

데이터 저장

RDS (PostgreSQL 17)

관리형 DB 로 상태 분리
SG 참조 접근 통제

관측·알람

Prometheus · Grafana · CloudWatch

지표 수집 · 대시보드
Slack 알람 전달

05. 애플리케이션 구성 - 인프라 전체를 지나가는 검증 단위

AI 텍스트 요약 API · FastAPI + Claude API + RDS · 단일 파일 88 줄

POST /summaries 한 요청의 여정



지킨 클라우드 네이티브 규약

- ✓ **stateless** 상태는 전부 RDS 에 . 파드는 언제 죽어도 되는 소모품
- ✓ **비밀값 0** 쿠버네티스 Secret 을 환경변수로 주입 . 코드 . 이미지 .repo 어디에도 없음
- ✓ **프로브 계약** /healthz 하나로 자가치유 판단과 배포 버전 확인을 같이 함

이 요청 하나가 통과하면 ALB 부터 RDS 까지 전 구간이 검증된 셈입니다

06. 시연 과정 (1/3) - GitOps 자동 배포

1

git push 버전 0.2.1 커밋

2

GitHub Actions OIDC 인증 → 이미지 빌드 → ECR 푸시

3

매니페스트 repo 이미지 태그 자동 커밋 (kustomize edit)

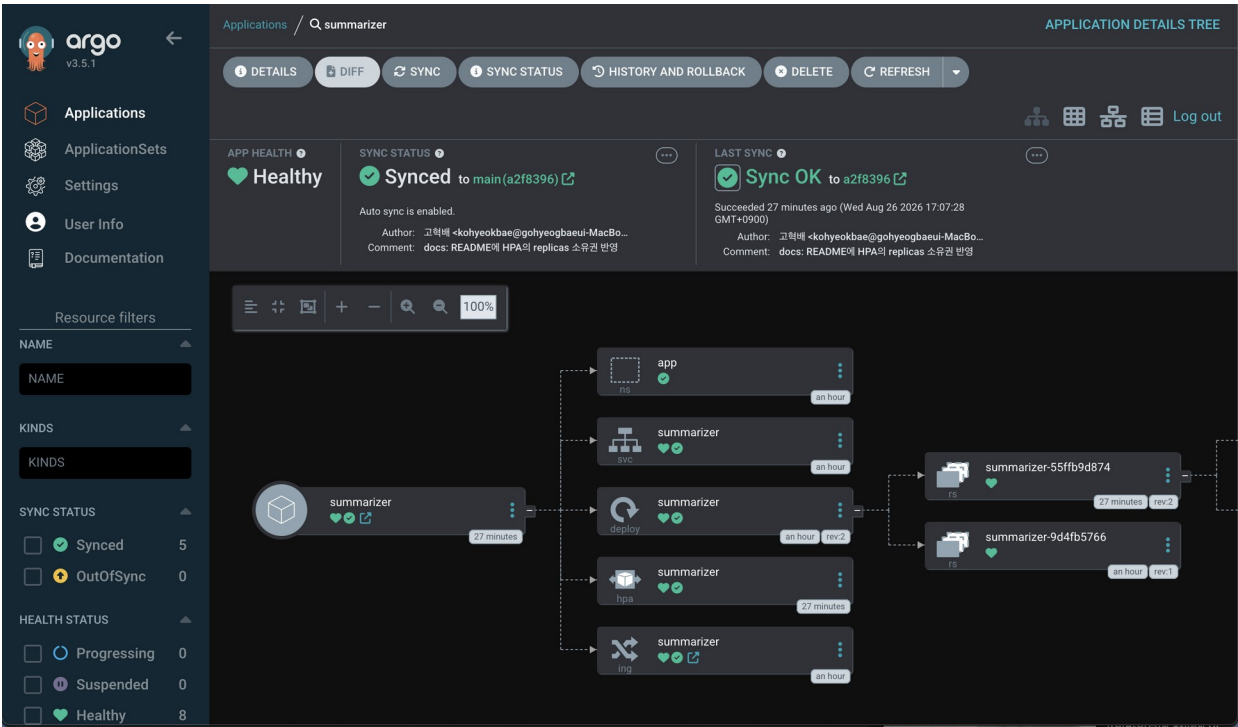
4

ArgoCD 변경 감지 → 롤링 배포

5

인터넷 검증 {"version": "0.2.1"} 응답 확인

ArgoCD 화면 : Synced / Healthy, hpa 까지 포함된 리소스 트리



- 앱 repo(공개) 와 매니페스트 repo(비공개) 를 분리 . Git 이 단일 진실 공급원
- drift 는 ArgoCD 가 감지해 Git 상태로 되돌리고 , 롤백은 git revert 한 번

06. 시연 과정 (2/3) - 관측·알람 검증



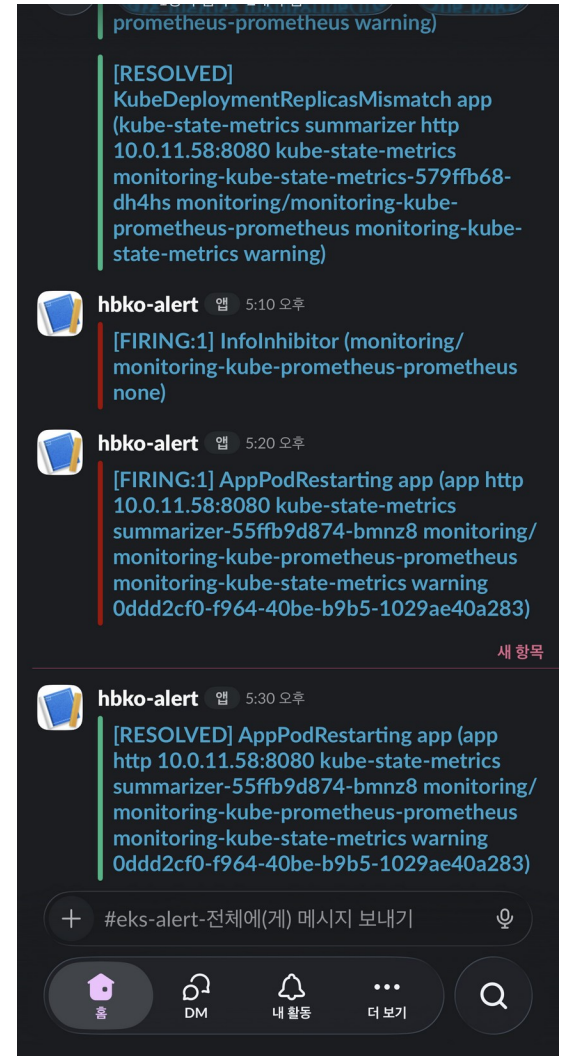
RDS 지표는 CloudWatch 에서 exporter 를 거쳐 Prometheus 로 모음 . 알람 경로를 하나로

장애를 직접 만들어서 확인

- 파드 강제 종료 → 알람 FIRING → Slack 수신 확인
- 복구되면 RESOLVED 통보까지 오는지 왕복 확인
- 파드 삭제 후 자동 복구 (self-healing) 도 같은 실험에서 확인
- 이후 Grafana OOMKilled 도 이 경로로 실제 이벤트가 잡힘

올리는 것을 확인한 알람만 믿기로 했습니다

Slack: FIRING → RESOLVED 수신

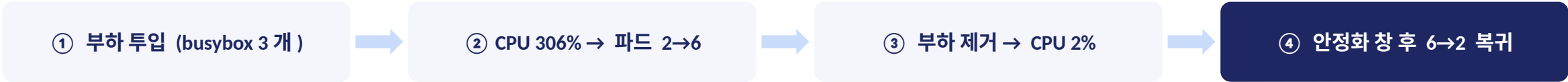
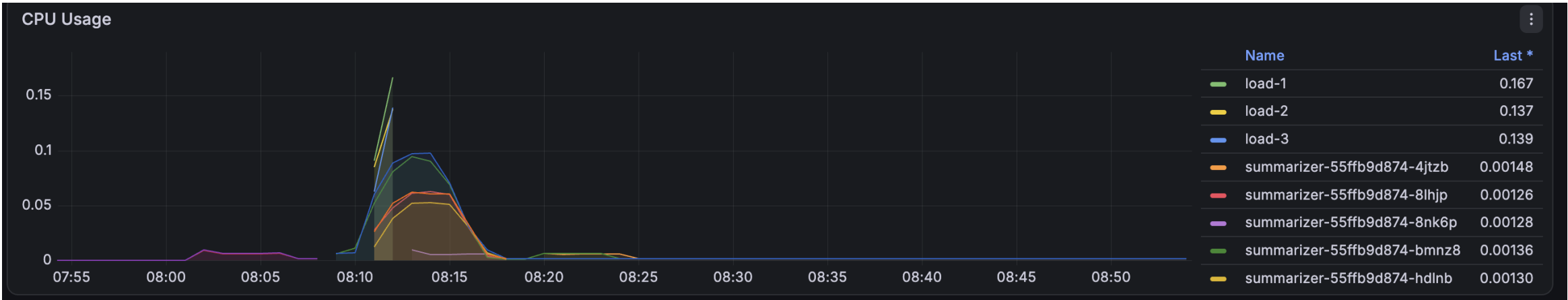


06. 시연 과정 (3/3) - HPA 오토스케일링

kubectl get hpa / get pods: CPU 50% 기준 2~6, 파드는 서로 다른 노드 (AZ) 에 분산

```
[kohyeokbae@gohyeogbaeui-MacBookAir scripts % kubectl get hpa -n app
NAME          REFERENCE                TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
summarizer    Deployment/summarizer     cpu: 2%/50%  2        6        2         30m
[kohyeokbae@gohyeogbaeui-MacBookAir scripts %
[kohyeokbae@gohyeogbaeui-MacBookAir scripts % kubectl get pods -n app -o wide
NAME                                READY  STATUS   RESTARTS  AGE  IP             NODE                                                    NOMINATED NODE  READINESS GATES
summarizer-55ffb9d874-4jtzp         1/1    Running  0         19m  10.0.10.72     ip-10-0-10-230.ap-northeast-2.compute.internal          <none>           <none>
summarizer-55ffb9d874-p6lhn         1/1    Running  0         30m  10.0.11.203    ip-10-0-11-33.ap-northeast-2.compute.internal          <none>           <none>
kohyeokbae@gohyeogbaeui-MacBookAir scripts %
```

Grafana CPU Usage: 부하를 넣자 스파이크, summarizer 파드가 6 개로 늘어남 (우측 범례)



관찰: 6 번째 파드는 Pending(Too many pods). 노드를 늘리는 건 Cluster Autoscaler 몹이랴 HPA 의 한계도 같이 봄 (백로그 근거)

07. 트러블슈팅 (1/3) - 노드그룹 무증상 실패

[문제]

EKS 노드그룹이 38 분 만에 CREATE_FAILED. 그동안 상태 API(health.issues) 는 비어 있었음

[진단]

인스턴스가 났는지부터 확인해 범위를 절반으로 줄임 . 기동 자체가 거부된 것이었고 , 사유는 ASG 활동 로그에만 적혀 있었음

[원인]

AWS Free Plan 계정의 인스턴스 타입 제약 . 코드가 아니라 계정 정책 문제

[해결] 타입을 바꿔 우회하는 대신 Paid Plan 으로 전환 . 코드는 그대로 두고 재시도해 성공

[배운점] 실패 사유가 적히는 곳은 따로 있다. API 가 조용하다고 정상인 게 아니다

07. 트러블슈팅 (2/3) - OIDC 인증 불일치

[문제]

CI 의 AWS 인증 (OIDC) 실패 . 에러는 Not authorized 한 줄뿐

[진단]

전파 지연 , 정책 연산자 두 가설을 세워 확인했지만 둘 다 아니었음 . CloudTrail 에 GitHub 가 실제로 보낸 값이 남아 있었음

[원인]

GitHub 가 sub 클레임에 @ 계정 ID·@ 저장소 ID 를 붙이는 새 형식을 쓰는데 , 커뮤니티 모듈은 옛 형식으로 조건을 만들고 있었음

[해결] Role 을 직접 정의하고 ID 까지 고정 . 계정명 탈취 · 재생성 공격까지 막히는 부수효과

[배운점] 신뢰 정책은 내가 기대하는 값이 아니라 상대가 실제로 보내는 값에 맞춰야 한다

07. 트러블슈팅 (3/3) - 파드 전면 기동 불가

[문제]

첫 배포에서 파드 전부가 CreateContainerConfigError. 이미지 풀과 Secret 은 정상인데 컨테이너 생성이 거부됨

[진단]

kubectl describe pod 의 Events 에 그대로 적혀 있었음 : runAsNonRoot and image has non-numeric user (appuser), cannot verify

[원인]

이미지의 USER 가 이름 (appuser) 뿐이라 kubelet 이 시작 전에 UID 를 알 수 없음 . 이미지도 매니페스트도 각자는 맞는데 조합이 검증이 안 됨

[해결] securityContext 에 runAsUser: 1000 을 명시해 재빌드 없이 해결 . Dockerfile 을 숫자 UID 로 바꾸는 건 백로그

[배운점] 보안 설정은 각각 맞는지가 아니라 조합해서 검증되는지까지 봐야 한다

07. 구축 후 재점검 - 에러 없는 문제 찾기

자동화 도구끼리의 충돌 사전 제거

HPA 가 늘린 파드를 ArgoCD 가 도로 줄일 수 있는 구조를 배포 전에 발견
→ replicas 를 Git 에서 빼고 소유권을 HPA 로

rollout 성공 ≠ 무중단

배포 성공 직후 응답 실패 1 회를 그냥 넘기지 않고 추적
→ 계층마다 완료 기준이 다르다는 걸 확인 . Readiness Gate 는 백로그

HPA 의 한계 실증

증설 중 파드 Pending(Too many pods). 노드 쪽 한계였음
→ Cluster Autoscaler 도입 근거

Grafana OOMKilled 실측

메모리 제한 256Mi 가 실사용에서는 부족했음
→ 512Mi 로 올리고 근거를 주석으로 코드에 남김

08. 핵심 설계 의사결정

결정 항목	선택	근거
배포 방식	ArgoCD (pull)	CI 가 직접 배포하는 방식과 달리 자격증명이 클러스터 밖으로 안 나감 . drift 감지 · 자동 복구 , Git 이 단일 진실 공급원
HPA 기준	CPU 50% · 파드 2~6	낮으면 과잉 증설 , 높으면 대응 지연 / 최소 2 = AZ 분산 유지 , 최대 6 = 비용 상한 . 대기형 부하는 CPU 가 안 오르는 함정을 검증에서 실확인
데이터 계층	RDS 관리형 (단일 AZ)	백업 · 패치를 관리형에 위임 , 앱과 수명주기 분리 / Multi-AZ 는 비용 약 2 배로 인지한 상태에서 제외
파드 상태	stateless (상태는 RDS 에)	파드를 언제든지 죽일 수 있어야 HPA 증감과 Spot 회수가 성립 . 세션 · 파일을 파드 안에 두지 않음
DB 버전	PostgreSQL 17	현업에서 운영 중인 솔루션 DB 와 같은 버전 . 재생성 구조라 코드 한 줄만 바꿈
네트워크 비용	단일 NAT Gateway	AZ 별 NAT 대비 월 \$40 절감 / 단일 장애점은 문서화하고 수용 (포트폴리오 환경)
비밀값 관리	K8s Secret	Secrets Manager 는 이 규모엔 과함 . 코드 · 이미지 · repo 에 비밀값 0 은 충족 . tfvars·state 평문 한계는 알고 있고 , 운영 전환 시 Secrets Manager 검토

09. 마무리 - 운영 경험이 설계에 어떻게 쓰였나

A. 고객 클라우드를 관제 · 운영하며 쌓인 판단 기준이 이 프로젝트의 설계 근거가 됐습니다

기존 운영 관점 (Before)

ZENIUS 관제 · KT/NHN/Naver Cloud 운영

- 남이 정한 임계값과 경보에 대응
- 고객 환경의 장애를 분석해 보고
- 정해진 절차로 서버 · DB · 백업 점검
- Docker 이미지로 환경 표준화 (제공 시간 40% 단축)

새로운 관점 (After)

이 프로젝트 - 직접 설계 · 검증

- 알람 임계값을 직접 판단 (HPA 50% · 재시작 감지)
- 장애를 직접 만들어 경보 경로 검증
- 재기동 9 단계 · 종료 4 단계 절차를 직접 작성
- 표준화를 파이프라인 자동화로 확장 (CI→ECR→ArgoCD)

09. 마무리 - 배운 점과 다음 단계

배운 점

- 완료 기준은 계층마다 다르다 : rollout 성공 ≠ LB 무중단 , HPA 증설 ≠ 노드 여유
- 실패 원인이 적히는 곳은 따로 있다 : ASG 로그 , CloudTrail, 파드 Events
- 문서가 곧 재현성이다 : ADR 3 건 , 트러블슈팅 7 건 , 운영 루틴으로 5 회 재현

다음 단계 (백로그)

- EKS 버전 상향 (1.31 → 1.34+): 확장 지원 요금 회피
- Cluster Autoscaler: Pending 관찰을 근거로 노드 자동 확장
- Pod Readiness Gate: 롤링 배포 무중단 보강
- 알람 라우팅 튜닝 (InfoInhibitor 등)
- 테스트 코드 + CI 테스트 단계 · 앱 고도화

감사합니다

관제하던 클라우드에서, 설계하는 클라우드로

코드 한 줄부터 회수까지, 전 과정을 직접 검증했습니다

트러블슈팅 7 건 (문제 → 진단 → 해결 → 배운 점) · ADR 3 건 · 작업 로그 · 재기동 / 종료 루틴



QR: GitHub repo

이메일 soc05272@naver.com

GitHub github.com/soc05272/eks-gitops-portfolio